

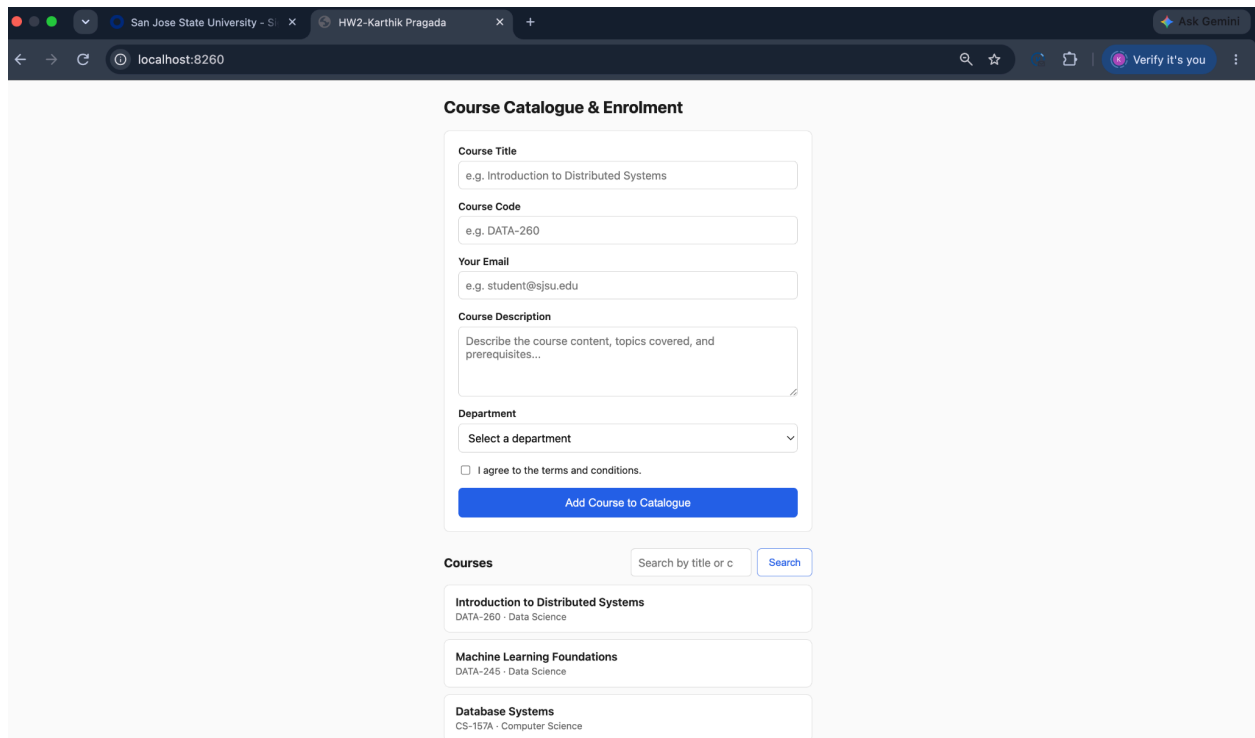
DATA - 260
HomeWork- 2
Name: Karthik Pragada
SJSU ID: 018768360

Value	Definition
SID	8360
PORT_BASE	8260
PREFIX	s8360
SEED	8360
VERIFY_SEED	268360
DOMAIN_ID	0
HARDWARE	MacBook Air, Apple M4, 16 GB RAM
Local Model Used	qwen3:8b Ollama Model
Tagged commit hash	d57ba70622ec96c8e760f6efa6bbfe4b6e4bb41d
Git repo	https://github.com/prags8585/data260-8360/tree/main

Part 1 (HTML, CSS)

Styled the Course Catalogue form and entity list so it stays usable at 375px width, and added visible loading, empty, and error states. Since the assignment's Part 2 language ("redirected to the home view showing the updated list") describes a server-rendered pattern, I rebuilt the page as a FastAPI + Jinja2 app instead of the static forms POST directly to the server and the server redirects back to a freshly rendered home page. This became the foundation Part 2 builds real data onto.

Desktop view



The screenshot shows a web browser window with the address bar at localhost:8260. The page title is "Course Catalogue & Enrolment". The form contains the following fields:

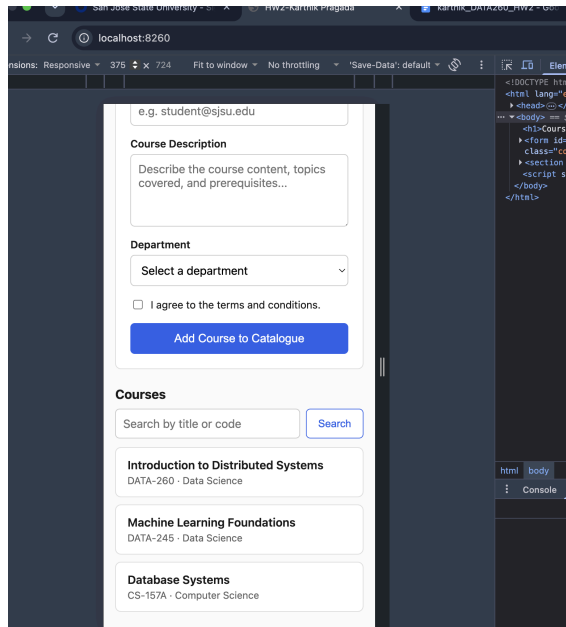
- Course Title**: e.g. Introduction to Distributed Systems
- Course Code**: e.g. DATA-260
- Your Email**: e.g. student@sjsu.edu
- Course Description**: Describe the course content, topics covered, and prerequisites...
- Department**: Select a department (dropdown menu)
- ☐ I agree to the terms and conditions.
- Add Course to Catalogue** (blue button)

Below the form, there is a section titled "Courses" with a search bar "Search by title or c" and a "Search" button. The list of courses is as follows:

- Introduction to Distributed Systems**
DATA-260 · Data Science
- Machine Learning Foundations**
DATA-245 · Data Science
- Database Systems**
CS-157A · Computer Science

Mobile view (375px width)

Verified the form, the Update/Delete cards, and the course list all remain fully usable at 375px nothing overflows, buttons stay full-width and tappable, and the two management cards stack vertically instead of squeezing side by side.



Empty state

Shown whenever the course list has zero results (either because no courses exist yet, or because a search matched nothing).

Course Catalogue & Enrolment

Course Title

Course Code

Your Email

Course Description

Describe the course content, topics covered, and prerequisites...

Department

Select a department ▼

☐ I agree to the terms and conditions.

Add Course to Catalogue

Update Course #1

Course Title

Course Code

Update Course #1

Error state

A red banner driven by a flash-style `?error=` query parameter. Part 2's create/update/delete routes redirect here with a message whenever something goes wrong (missing required fields, deleting an empty list, etc.) instead of crashing.

Course Catalogue & Enrolment

Course not found. It may have already been deleted.

Course Title

e.g. Introduction to Distributed System

Course Code

e.g. DATA-260

Your Email

e.g. student@sjsu.edu

Course Description

Describe the course content, topics covered, and prerequisites...

Department

Select a department

Loading state

The instant any form is submitted, a small script shows a "Loading..." indicator before the browser finishes navigating to the new page. The create route also has a short artificial delay so this is genuinely visible instead of flashing by instantly on a fast local server.

Search by title or code

Search

Loading...

Introduction to Distributed Systems

DATA-260 · Data Science

Machine Learning Foundations

DATA-245 · Data Science

Database Systems

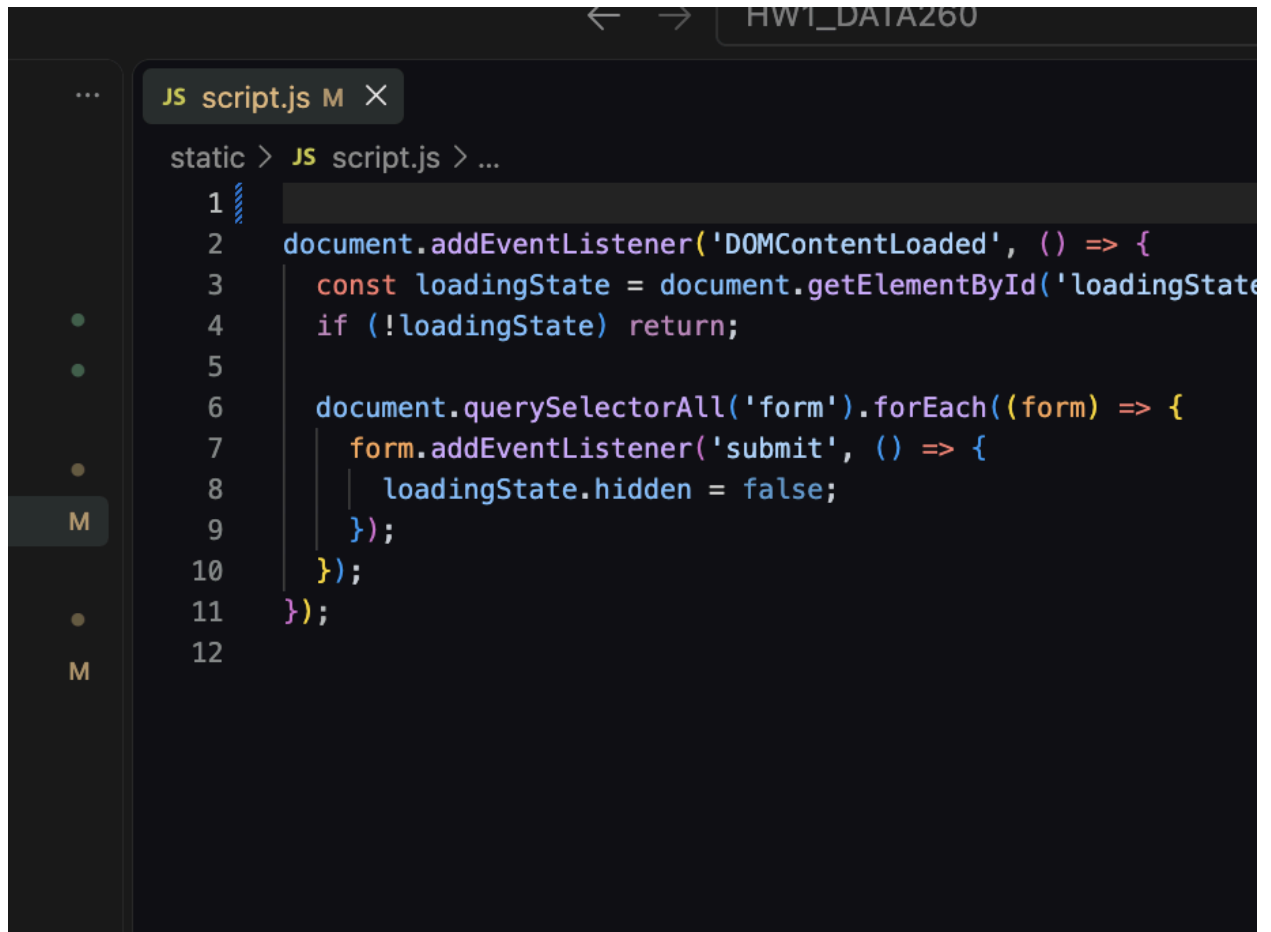
CS-157A · Computer Science

Code - Index.html

```
HW1_DATA260

< index.html M X
templates > < index.html > ...
2  <html lang="en">
9  <body>
16  <form id="courseForm" method="POST" action="/courses" class="course-form">
72  <div class="field checkbox-field">
73  <input type="checkbox" id="agreeTerms" name="agreeTerms">
74  <label for="agreeTerms">I agree to the terms and conditions.</label>
75  </div>
76
77  <button type="submit">Add Course to Catalogue</button>
78 </form>
79
80 <section class="manage-section">
81   <div class="manage-card">
82     <h2>Update Course #1</h2>
83     {% if course_1 %}
84     <form method="POST" action="/courses/1/update" class="inline-form">
85       <div class="field">
86         <label for="update1Title">Course Title</label>
87         <input type="text" id="update1Title" name="courseTitle" value="{{ course_1.courseTitle }}" required="">
88       </div>
89       <div class="field">
90         <label for="update1Code">Course Code</label>
91         <input type="text" id="update1Code" name="courseCode" value="{{ course_1.courseCode }}" required="">
92       </div>
93       <button type="submit">Update Course #1</button>
94     </form>
95     {% else %}
96     <p class="state-banner empty-state">No course with ID 1 exists to update.</p>
97     {% endif %}
98   </div>
99
100   <div class="manage-card">
101     <h2>Delete Highest-ID Course</h2>
102     <p class="manage-hint">Removes whichever course currently has the highest ID.</p>
103     <form method="POST" action="/courses/delete-highest" class="inline-form">
104       <button type="submit" class="danger-button">Delete Highest-ID Course</button>
105     </form>
106   </div>
107 </section>
108
109 <section class="list-section">
110   <div class="list-header">
111     <h2>Courses</h2>
112     <form method="GET" action="/" class="search-form">
113       <input type="search" name="q" placeholder="Search by title or code" value="{{ q or '' }}">
114       <button type="submit">Search</button>
115     </form>
116   </div>
```

Script .js

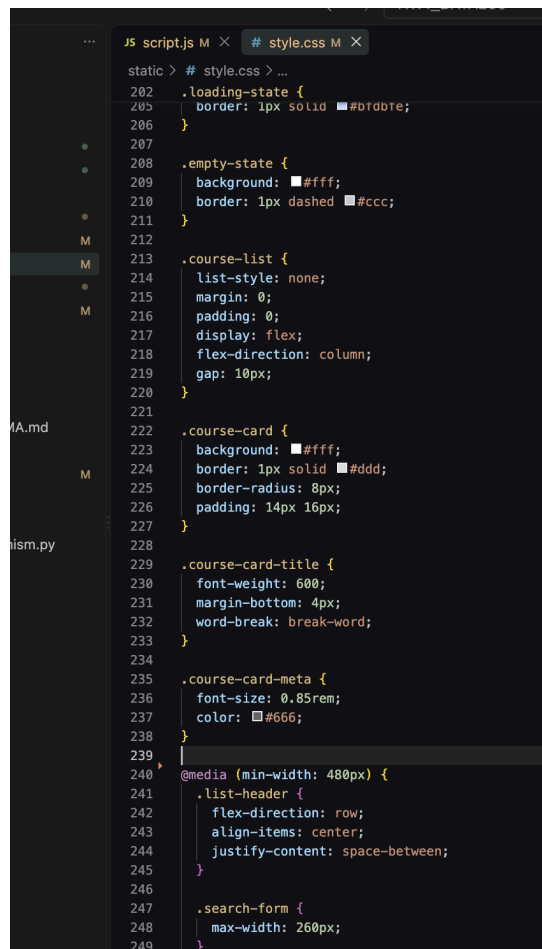


The image shows a code editor window with a dark theme. At the top, there's a tab labeled 'JS script.js M' with a close button. Below the tab, the code is as follows:

```
static > JS script.js > ...  
1  
2 document.addEventListener('DOMContentLoaded', () => {  
3     const loadingState = document.getElementById('loadingState');  
4     if (!loadingState) return;  
5  
6     document.querySelectorAll('form').forEach((form) => {  
7         form.addEventListener('submit', () => {  
8             loadingState.hidden = false;  
9         });  
10    });  
11 });  
12
```

The left sidebar shows a file explorer with a folder named 'M' containing several files, some of which are marked with green dots.

Style.css



```
static > # style.css > ...
202 .loading-state {
203   border: 1px solid #d1dbf6;
204 }
205
206
207
208 .empty-state {
209   background: #fff;
210   border: 1px dashed #ccc;
211 }
212
213 .course-list {
214   list-style: none;
215   margin: 0;
216   padding: 0;
217   display: flex;
218   flex-direction: column;
219   gap: 10px;
220 }
221
222 .course-card {
223   background: #fff;
224   border: 1px solid #ddd;
225   border-radius: 8px;
226   padding: 14px 16px;
227 }
228
229 .course-card-title {
230   font-weight: 600;
231   margin-bottom: 4px;
232   word-break: break-word;
233 }
234
235 .course-card-meta {
236   font-size: 0.85rem;
237   color: #666;
238 }
239
240 @media (min-width: 480px) {
241   .list-header {
242     flex-direction: row;
243     align-items: center;
244     justify-content: space-between;
245   }
246
247   .search-form {
248     max-width: 260px;
249   }
```

Part 2 (FastAPI)

Built a FastAPI backend on `PORT_BASE` (8260) with real in-memory CRUD for the Course entity, replacing Part 1's mock data. All four routes redirect back to the home view showing the updated list, matching the assignment's Post/Redirect/Get pattern.

1. Add a new course

Filled out the form (title, code, email, description, department, terms checkbox) and submitted. The new course is appended and appears at the end of the list after the redirect.

Before:

localhost:8260

Course Code

Your Email

Course Description

Department

☐ I agree to the terms and conditions.

Add Course to Catalogue

Update Course #1
No course with ID 1 exists to update.

Delete Highest-ID Course
Removes whichever course currently has the highest ID.
Delete Highest-ID Course

Courses **Search**

Introduction to Distributed Systems
DATA-260 · Data Science

big data technologies
data228 · Computer Science

Data Systems
CS-157A · Computer Science

After: Adding datawarehouse data 226

☐ I agree to the terms and conditions.

Add Course to Catalogue

Update Course #1
No course with ID 1 exists to update.

Delete Highest-ID Course
Removes whichever course currently has the highest ID.
Delete Highest-ID Course

Courses **Search**

Introduction to Distributed Systems
DATA-260 · Data Science

big data technologies
data228 · Computer Science

Data Systems
CS-157A · Computer Science

Data warehouse
data 226 · Data Science

2. Update the record with ID 1

A small "Update Course #1" form, pre-filled with whatever course #1 currently holds, lets me submit new title/code values specifically for that record.

Before (course #1 is "Introduction to Distributed Systems / DATA-260"):

127.0.0.1:8260

Department

Select a department

☐ I agree to the terms and conditions.

Add Course to Catalogue

Update Course #1

Course Title

Introduction to Distributed System

Course Code

DATA-260

Update Course #1

Delete Highest-ID Course

Removes whichever course currently has the highest ID.

Delete Highest-ID Course

Courses

Search by title or c

Introduction to Distributed Systems

DATA-260 · Data Science

After (updated to " Deep Learning / Data 255"):

☐ I agree to the terms and conditions.

Add Course to Catalogue

Update Course #1

Course Title

Deep Learning

Course Code

DATA-255

Update Course #1

Delete Highest-ID Course

Removes whichever course currently has the highest ID.

Delete Highest-ID Course

Courses

Search by title or c

Search

Deep Learning

DATA-255 · Data Science

3. Delete the record with the highest ID

The "Delete Highest-ID Course" button always removes whichever record currently has the maximum ID : at this point that's "**datawarehouse data 226**" (ID 5, the one just created in step 1).

127.0.0.1:8260

Update Course #1

Course Title

Deep Learning

Course Code

DATA-255

Update Course #1

Delete Highest-ID Course

Removes whichever course currently has the highest ID.

Delete Highest-ID Course

Courses

Search by title or c

Search

Deep Learning

DATA-255 · Data Science

Machine Learning Foundations

DATA-245 · Data Science

Data Systems

CS-157A · Computer Science

big data technologies

data228 · Data Science

Data warehouse

data 226 · Data Science

After (Data warehouse removed):

127.0.0.1:8260

Add Course to Catalogue

Update Course #1

Course Title

Deep Learning

Course Code

DATA-255

Update Course #1

Delete Highest-ID Course

Removes whichever course currently has the highest ID.

Delete Highest-ID Course

Courses

Search by title or c

Search

Deep Learning

DATA-255 · Data Science

Machine Learning Foundations

DATA-245 · Data Science

Data Systems

CS-157A · Computer Science

big data technologies

data228 · Data Science

4. Search by primary or secondary field

Typing into the search box and submitting filters the list to only courses whose title or code contains the query (case-insensitive). Search "Machine" - matches all remaining courses (via title or course code):

The screenshot shows a web interface for managing courses. On the left, there are two input fields: 'Deep Learning' and 'Course Code' with the value 'DATA-255'. Below these is a blue button labeled 'Update Course #1'. To the right of these fields is a blue button labeled 'Delete Highest-ID Course'. Below the input fields, there is a section titled 'Courses'. It contains a search input field with the text 'Machine' and a blue button labeled 'Search'. Below the search results, there is a box containing the text 'Machine Learning Foundations' and 'DATA-245 · Data Science'.

Search with no matches : falls back to the empty state

The screenshot shows the same web interface as before, but with a search for 'gen ai'. The search input field now contains 'gen ai' and the 'Search' button is still present. The search results box is now empty, displaying the message 'No courses match "gen ai".' in a dashed border. The 'Update Course #1' and 'Delete Highest-ID Course' buttons remain visible on the left side of the interface.

Code - main.py

```
JS script.js M X # style.css M X main.py M X
main.py
77
78 @app.post("/courses")
79 async def create_course(
80     courseTitle: str = Form(...),
81     courseCode: str = Form(...),
82     email: str = Form(""),
83     description: str = Form(""),
84     department: str = Form(""),
85     agreeTerms: str | None = Form(None),
86 ):
87     if not courseTitle.strip() or not courseCode.strip():
88         return RedirectResponse(url="/?error=Course+title+and+code+are+required.", status_code=303)
89
90     await asyncio.sleep(1.5)
91
92     COURSES.append({
93         "id": _get_next_id(),
94         "courseTitle": courseTitle.strip(),
95         "courseCode": courseCode.strip(),
96         "email": email.strip(),
97         "description": description.strip(),
98         "department": department,
99         "agreeTerms": bool(agreeTerms),
100     })
101     return RedirectResponse(url="/", status_code=303)
102
103
104 @app.post("/courses/1/update")
105 async def update_course_1(courseTitle: str = Form(...), courseCode: str = Form(...)):
106     course = _find_by_id(1)
107     if course is None:
108         return RedirectResponse(url="/?error=Course+with+ID+1+not+found.", status_code=303)
109     if not courseTitle.strip() or not courseCode.strip():
110         return RedirectResponse(url="/?error=Course+title+and+code+are+required.", status_code=303)
111
112     course["courseTitle"] = courseTitle.strip()
113     course["courseCode"] = courseCode.strip()
114     return RedirectResponse(url="/", status_code=303)
115
116
117 @app.post("/courses/delete-highest")
118 async def delete_highest_course():
119     if not COURSES:
120         return RedirectResponse(url="/?error=No+courses+to+delete.", status_code=303)
121
122     highest = max(COURSES, key=lambda c: c["id"])
123     COURSES.remove(highest)
124     return RedirectResponse(url="/", status_code=303)
```

Part 3 (Stateful Agent Graph)

Refactored HW1's sequential Planner -> Reviewer -> Finalizer pipeline (`agents_demo.py`) into a LangGraph StateGraph (`agent_graph.py`) built around a supervisor node that re-evaluates where to go next after every node finishes, so the graph can loop back to the Planner for self-correction instead of always running straight through. A deterministic `router_logic()` function (no LLM call) inspects the shared state on each supervisor visit and decides the next node: it checks a `turn_count` ceiling first (a safety valve against runaway loops), then whether a proposal exists yet, then whether that proposal has cleared the Part 4 schema gate, then whether the Reviewer has flagged issues. Both LLM-calling nodes (Planner, Reviewer) go through `ModelClient` from HW1 Part 4; they never touch `ChatOllama/LangChain` directly.

1. Baseline run

Default demo input (distributed systems course), `qwen3:8b`, temperature 0.7. The Planner proposes 3 tags and a summary, the proposal passes the Part 4 schema gate, and the Reviewer approves it on the first pass (`"has_issues": false`), so the router sends control straight to END: 4 supervisor visits, 1 planner call, 0 retries, not abandoned.

```

(base) prags@karthiks-MacBook-Air ~ % cd /Users/prags/Desktop/HW1_DATA260
source .venv/bin/activate
ollama list
NAME          ID          SIZE      MODIFIED
qwen3:8b      500a1f067a9f 5.2 GB    2 weeks ago
codellama:7b  8fd8f8f752f6e 3.8 GB    7 months ago
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % python agent_graph.py --temperature 0.7
--- ROUTER: no proposal yet -> planner ---
=== after node: supervisor ===
--- NODE: Planner ---
{
  "tags": [
    "key-value store",
    "leader election protocol",
    "node failures"
  ],
  "summary": "This course teaches design and implementation of distributed systems with a focus on fault tolerance and consensus algorithms."
}
=== after node: planner ===
--- ROUTER: proposal not yet schema-validated -> validate ---
=== after node: supervisor ===
--- NODE: Validate (Pydantic) ---
valid
=== after node: validate ===
--- ROUTER: schema OK, not yet reviewed -> reviewer ---
=== after node: supervisor ===
--- NODE: Reviewer ---
{
  "tags": [
    "key-value store",
    "leader election protocol",
    "node failures"
  ],
  "summary": "This course teaches design and implementation of distributed systems with a focus on fault tolerance and consensus algorithms.",
  "has_issues": false,
  "feedback": ""
}
=== after node: reviewer ===
--- ROUTER: no issues -> END ---
=== after node: supervisor ===

=====
FINALIZED (Publish)
=====
{
  "tags": [
    "key-value store",
    "leader election protocol",
    "node failures"
  ],
  "summary": "This course teaches design and implementation of distributed systems with a focus on fault tolerance and consensus algorithms."
}

Total turns (supervisor visits): 4
Planner calls: 1 (retries: 0)
Abandoned at ceiling: False
Latency: 10820 ms
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %

```

2. Forcing the self-correction loop

reviewer_node has a `--force-issue` testing hook: it overrides whatever the model actually said and always reports `has_issues: true`, so the reviewer -> planner loop-back edge can be proven to fire on demand rather than assuming it works from a single lucky run.

This run also passes a deliberately tight `--ceiling 4` to exercise the turn-ceiling safety valve at the same time: the Planner drafts, the Reviewer force-rejects it, the router sends control back to the Planner for a revised draft but, by the time that second Planner call finishes, `turn_count` has climbed to 5, which exceeds `ceiling=4`, so the router ends the graph before the revised draft is ever re-validated or re-reviewed. The deterministic `finalize()` function (a non-LLM safety net, same idea as HW1's Finalizer) still publishes the best draft available the unreviewed revision instead of crashing or returning nothing, which is exactly the abandoned-at-ceiling case it exists to handle.

```

((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % python agent_graph.py --temperature 0.7 --force-issue --ceiling 4
--- ROUTER: no proposal yet -> planner ---
=== after node: supervisor ===
--- NODE: Planner ---
{
  "tags": [
    "consensus algorithms",
    "key-value store",
    "leader election"
  ],
  "summary": "A course on designing distributed systems with focus on consensus, fault tolerance, and leader election protocols."
}
=== after node: planner ===
--- ROUTER: proposal not yet schema-validated -> validate ---
=== after node: supervisor ===
--- NODE: Validate (Pydantic) ---
valid
=== after node: validate ===
--- ROUTER: schema OK, not yet reviewed -> reviewer ---
=== after node: supervisor ===
--- NODE: Reviewer ---
{
  "tags": [
    "consensus algorithms",
    "key-value store",
    "leader election"
  ],
  "summary": "A course on designing distributed systems with focus on consensus, fault tolerance, and leader election protocols.",
  "has_issues": true,
  "feedback": "(forced for testing the correction loop)"
}
=== after node: reviewer ===
--- ROUTER: issues found -> planner (self-correction) ---
=== after node: supervisor ===
--- NODE: Planner ---
{
  "tags": [
    "distributed systems design",
    "fault tolerance implementation",
    "leader election protocol"
  ],
  "summary": "A course on designing distributed systems with focus on consensus, fault tolerance, and leader election protocols."
}
=== after node: planner ===
--- ROUTER: turn ceiling (4) reached -- ending ---
=== after node: supervisor ===

```

```

=====
FINALIZED (Publish)
=====
{
  "tags": [
    "distributed systems design",
    "fault tolerance implementation",
    "leader election protocol"
  ],
  "summary": "A course on designing distributed systems with focus on consensus, fault tolerance, and leader election protocols."
}

Total turns (supervisor visits): 5
Planner calls: 2 (retries: 1)
Abandoned at ceiling: True
Latency: 7851 ms
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % █

```

Code - agent_graph.py (router + supervisor loop)

```
JS script.js M X # style.css M X main.py M X agent_graph.py U X
agent_graph.py
142 def supervisor_node(state: AgentState) -> Dict[str, Any]:
146
147
148 def router_logic(state: AgentState) -> str:
149     if state.get("turn_count", 0) > TURN_CEILING_DEFAULT:
150         print(f"--- ROUTER: turn ceiling ({TURN_CEILING_DEFAULT}) reached -- ending ---")
151         return END
152     if not state.get("planner_proposal"):
153         print("--- ROUTER: no proposal yet -> planner ---")
154         return "planner"
155     if not state.get("schema_checked"):
156         print("--- ROUTER: proposal not yet schema-validated -> validate ---")
157         return "validate"
158     feedback = state.get("reviewer_feedback")
159     if feedback and feedback.get("has_issues"):
160         print("--- ROUTER: issues found -> planner (self-correction) ---")
161         return "planner"
162     if not feedback:
163         print("--- ROUTER: schema OK, not yet reviewed -> reviewer ---")
164         return "reviewer"
165     print("--- ROUTER: no issues -> END ---")
166     return END
167
168
169 def build_graph():
170     graph = StateGraph(AgentState)
171     graph.add_node("supervisor", supervisor_node)
172     graph.add_node("planner", planner_node)
173     graph.add_node("validate", validate_node)
174     graph.add_node("reviewer", reviewer_node)
175
176     graph.set_entry_point("supervisor")
177     graph.add_conditional_edges(
178         "supervisor",
179         router_logic,
180         {"planner": "planner", "validate": "validate", "reviewer": "reviewer", END: END},
181     )
182     graph.add_edge("planner", "supervisor")
183     graph.add_edge("validate", "supervisor")
184     graph.add_edge("reviewer", "supervisor")
185
186     return graph.compile()
187
188
189 def finalize(state: Dict[str, Any]) -> Dict[str, Any]:
190     source = state.get("reviewer_feedback") or state.get("planner_proposal") or {}
191     tags = list(source.get("tags", []))[:3]
192     while len(tags) < 3:
193         tags.append("general")
194     summary = str(source.get("summary", "")).strip()
195     words = summary.split()
196     if len(words) > 25:
197         summary = " ".join(words[:25])
```

Part 4

Added a deterministic Pydantic validation gate (validate_node / TagsSummarySchema) between the Planner and Reviewer: exactly 3 tags, each 3–30 characters, summary at most 25 words. A schema failure is turned into a synthetic reviewer_feedback-shaped rejection and routed through the exact same has_issues loop-back path as a real Reviewer complaint, so the Planner sees one consistent retry mechanism whether the correction came from the deterministic gate or from the LLM reviewer.

1. Fixed input, ceiling = 6

Input: reports/hw02/cases/schema_input.json (a cloud-computing/Kubernetes course, chosen to be easy to tag precisely in 3 terms). The first proposal clears the schema gate and the Reviewer approves it immediately - 4 supervisor visits, 1 planner call, 0 retries.

```
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % python agent_graph.py --input reports/hw02/cases/schema_input.json --temperature 0.7
--- ROUTER: no proposal yet -> planner ---
=== after node: supervisor ===
--- NODE: Planner ---
{
  "tags": [
    "cloud computing architectures",
    "kubernetes orchestration",
    "microservices design"
  ],
  "summary": "This course covers cloud computing, distributed systems, and Kubernetes for scalable application deployment on AWS and GCP."
}
=== after node: planner ===
--- ROUTER: proposal not yet schema-validated -> validate ---
=== after node: supervisor ===
--- NODE: Validate (Pydantic) ---
valid
=== after node: validate ===
--- ROUTER: schema OK, not yet reviewed -> reviewer ---
=== after node: supervisor ===
--- NODE: Reviewer ---
{
  "tags": [
    "cloud computing architectures",
    "kubernetes orchestration",
    "microservices design"
  ],
  "summary": "This course covers cloud computing, distributed systems, and Kubernetes for scalable application deployment on AWS and GCP.",
  "has_issues": false,
  "feedback": ""
}
=== after node: reviewer ===
--- ROUTER: no issues -> END ---
=== after node: supervisor ===

=====
FINALIZED (Publish)
=====
{
  "tags": [
    "cloud computing architectures",
    "kubernetes orchestration",
    "microservices design"
  ],
  "summary": "This course covers cloud computing, distributed systems, and Kubernetes for scalable application deployment on AWS and GCP."
}

Total turns (supervisor visits): 4
Planner calls: 1 (retries: 0)
Abandoned at ceiling: False
Latency: 6452 ms
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

2. Adversarial input, ceiling = 6

Input: reports/hw02/cases/adversarial_input.json, a course description deliberately written to span ten unrelated CS subfields in one semester hard to compress into 3 accurate, specific tags. The first proposal is mechanically valid (passes the schema gate) but the Reviewer rejects it as too narrow for the course's breadth ("too specific and do not reflect the broader scope"), so the Planner revises toward broader, more general tags. This time the Reviewer is satisfied ("has_issues": false) but that approval comes on the graph's 7th supervisor visit, and router_logic() checks the turn ceiling before it ever looks at the Reviewer's feedback. Since

turn_count=7 > ceiling=6, the router ends the run on the ceiling check without ever reaching the branch that would have read has_issues: false and _sent it to END normally. The published output ends up identical either way (finalize() just republishes the same already-approved draft), but the run is still logged as abandoned_at_ceiling by the harness's turn_count > ceiling definition a case where "abandoned" is a bookkeeping artifact of check ordering, not a sign the content is unfinished or wrong.

```
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % python agent_graph.py --input reports/hw02/cases/adversarial_input.json --temperature 0.7 --ceiling 6
--- ROUTER: no proposal yet -> planner ---
=== after node: supervisor ===
--- NODE: Planner ---
{
  "tags": [
    "computer networking",
    "machine learning",
    "natural language processing"
  ],
  "summary": "A comprehensive course covering ten computer science subfields with weekly guest lectures and a final exam."
}
=== after node: planner ===
--- ROUTER: proposal not yet schema-validated -> validate ---
=== after node: supervisor ===
--- NODE: Validate (Pydantic) ---
valid
=== after node: validate ===
--- ROUTER: schema OK, not yet reviewed -> reviewer ---
=== after node: supervisor ===
--- NODE: Reviewer ---
{
  "tags": [
    "computer networking",
    "machine learning",
    "natural language processing"
  ],
  "summary": "A comprehensive course covering ten computer science subfields with weekly guest lectures and a final exam.",
  "has_issues": true,
  "feedback": "Tags are too specific and do not reflect the broader scope of the course."
}
=== after node: reviewer ===
--- ROUTER: issues found -> planner (self-correction) ---
=== after node: supervisor ===
--- NODE: Planner ---
{
  "tags": [
    "computer science survey",
    "multi-subfield course",
    "comprehensive final"
  ],
  "summary": "A semester-long course covering ten computer science areas with guest lectures and a comprehensive final exam."
}
=== after node: planner ===
--- ROUTER: proposal not yet schema-validated -> validate ---
=== after node: supervisor ===
--- NODE: Validate (Pydantic) ---
valid
=== after node: validate ===
--- ROUTER: schema OK, not yet reviewed -> reviewer ---
=== after node: supervisor ===
--- NODE: Reviewer ---
{
  "tags": [
    "computer science survey",
    "multi-subfield course",
    "comprehensive final"
  ],
  "summary": "A semester-long course covering ten computer science areas with guest lectures and a comprehensive final exam.",
  "has_issues": false,
  "feedback": ""
}
=== after node: reviewer ===
--- ROUTER: turn ceiling (6) reached -- ending ---
=== after node: supervisor ===

=====
FINALIZED (Publish)
=====
{
  "tags": [
    "computer science survey",
    "multi-subfield course",
    "comprehensive final"
  ],
  "summary": "A semester-long course covering ten computer science areas with guest lectures and a comprehensive final exam."
}

Total turns (supervisor visits): 7
Planner calls: 2 (retries: 1)
Abandoned at ceiling: True
Latency: 12224 ms
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

3. Batch experiments (run_schema_experiment.py)

Beyond the single demo runs above, runschema_experiment.py automates three repeated experiments through agent_graph.py's run() function (all with qwen3:8b, temperature 0.7) and writes per-run records to reports/hw02/raw/ plus a rolled-up summary to reports/hw02/raw/part4_summary.json. Each run is classified by classify() as Valid_first_attempt, valid_after_1_retry, valid_after_2plus_retries, or abandoned_at_ceiling (per the run() function's own turn_count > ceiling definition of "abandoned").

Experiment	n	Completion rate	Mean latency	Classification breakdown
Fixed input, ceiling=6 (4.3)	30	100%	5422 ms	30/30 valid_first_attempt
Fixed input, ceiling=2 (4.4)	20	0%	2483 ms	20/20 abandoned_at_ceiling
Fixed input, ceiling=10 (4.4)	20	100%	5856 ms	20/20 valid_first_attempt
Adversarial input, ceiling=6 (4.5)	5	0%	14778 ms	5/5 abandoned_at_ceiling (every run: 1 retry, turn_count=7)

4.3 - : retry behavior on a fixed, well-posed input: across 30 independent runs on schema_input.json, the Planner's first draft always cleared both the schema gate and the Reviewer every run finished in exactly 4 supervisor visits with 0 retries. The self-correction loop exists for when it's needed, but on an input this unambiguous it never actually fires.

4.4- ceiling comparison isolates a structural effect, not a model-quality one: the minimum number of supervisor visits for any successful run is 4 (planner -> validate -> reviewer -> end-check). At ceiling=2, the router's ceiling check fires on the 3rd supervisor visit (turn_count=3 > 2) before the Reviewer ever gets to run so all 20 runs were abandoned regardless of how good the Planner's draft was, with a correspondingly low mean latency (2483 ms, since the graph is cut short). At ceiling=10 every run completed normally in 4 visits, identical in shape to the ceiling=6 baseline (5856ms vs. 5422 ms, within normal LLM-latency noise). This shows the ceiling parameter has to be set with the graph's own minimum path length in mind, independent of how reliable the model is.

4.5 - the adversarial input reliably burns its one allotted retry: all 5 runs against adversarial_input.json needed exactly one self-correction retry, which puts the second Reviewer verdict at turn count=7. Because router_logic() checks the turn ceiling before it ever inspects the Reviewer's feedback, that 7th visit is caught by the ceiling=6 check regardless of whether the second review actually approved the revision or not so all 5 runs are logged as abandoned_at_ceiling under the script's strict turn_count > ceiling definition, with mean latency (14778 ms) roughly 2.7× the fixed-input baseline from the extra Planner/Reviewer round trip. As the single demo run above shows concretely (second review: "has_issues": false, yet still abandoned), "abandoned" here doesn't necessarily mean the published output is bad or unfinished it can just mean the Reviewer's approval arrived one supervisor visit later than the ceiling check that decides whether anyone gets to read it.

Code : agent_graph.py

```
JS script.js M × # style.css M × main.py M × agent_graph.py U ●
agent_graph.py
22 feedback, and return improved tags/summary."""
23
24 TURN_CEILING_DEFAULT = 6
25
26
27 class TagsSummarySchema(BaseModel):
28
29     tags: List[str]
30     summary: str
31
32     @field_validator("tags")
33     @classmethod
34     def check_tags(cls, v: List[str]) -> List[str]:
35         if len(v) != 3:
36             raise ValueError(f"must have exactly 3 tags, got {len(v)}")
37         for tag in v:
38             if not (3 <= len(tag) <= 30):
39                 raise ValueError(f"tag {tag!r} must be 3-30 characters long (got {len(tag)})")
40         return v
41
42     @field_validator("summary")
43     @classmethod
44     def check_summary(cls, v: str) -> str:
45         word_count = len(v.split())
46         if word_count > 25:
47             raise ValueError(f"summary must be at most 25 words (got {word_count})")
48         return v
49
50
51 class AgentState(TypedDict):
52     title: str
53     content: str
54     email: str
55     strict: bool
56     task: str
57     llm: Any
58     planner_proposal: Optional[Dict[str, Any]]
59     reviewer_feedback: Optional[Dict[str, Any]]
60     schema_checked: bool
61     turn_count: int
62     planner_calls: int
63     force_issue: bool
64
65
66 def planner_node(state: AgentState) -> Dict[str, Any]:
67     print("---- NODE: Planner ----")
68     llm: ModelClient = state["llm"]
69     user_prompt = f"Title: {state['title']}\n\nContent: {state['content']}"
70
71     feedback = state.get("reviewer_feedback")
72     if feedback:
73         user_prompt += (
```

Code - run_schema_experiment.py

```
JS script.js M × # style.css M × main.py M × agent_graph.py U • run_schema_experiment.py U ×
run_schema_experiment.py
42 def classify(record):
49     return "valid_after_2plus_retries"
50
51
52 def run_batch(title, content, n, ceiling, label):
53     records = []
54     for i in range(1, n + 1):
55         print(f"[{label}] run {i}/{n} (ceiling={ceiling})", flush=True)
56         try:
57             out = run(title, content, MODEL, TEMPERATURE, ceiling, force_issue=False, verbose=False)
58             rec = {
59                 "run_index": i,
60                 "ceiling": ceiling,
61                 "retries": out["retries"],
62                 "abandoned": out["abandoned"],
63                 "turn_count": out["turn_count"],
64                 "latency_ms": round(out["latency_ms"], 1),
65                 "final_tags": out["result"]["tags"],
66                 "final_summary": out["result"]["summary"],
67                 "error": None,
68             }
69             rec["classification"] = classify(rec)
70         except Exception as exc:
71             rec = {
72                 "run_index": i, "ceiling": ceiling, "retries": None, "abandoned": None,
73                 "turn_count": None, "latency_ms": None, "final_tags": None,
74                 "final_summary": None, "classification": "error", "error": str(exc),
75             }
76         print(json.dumps(rec), flush=True)
77         records.append(rec)
78     return records
79
80
81 def save_raw(records, filename):
82     path = RAW_DIR / filename
83     path.write_text(json.dumps(records, indent=2))
84     print(f"Saved {len(records)} records to {path}", flush=True)
85
86
87 def summarize(records, ceiling_label):
88     valid = [r for r in records if r["classification"] != "error"]
89     n = len(valid)
90     completed = [r for r in valid if not r["abandoned"]]
91     completion_rate = len(completed) / n if n else 0
92     latencies = [r["latency_ms"] for r in valid if r["latency_ms"] is not None]
93     mean_latency = statistics.mean(latencies) if latencies else None
94     counts = {}
95     for r in valid:
96         counts[r["classification"]] = counts.get(r["classification"], 0) + 1
97     return {
98         "ceiling": ceiling_label,
99         "n": n,
100         "completion_rate": round(completion_rate, 3),
```